BC01-FM04 · séance 3 · mardi 10 novembre 2026, matin

La liste d'emprunts · fiche solo

À rendre : `NOM-Prenom-React-S3.zip`, le vendredi 13 novembre à 18 h 00
Pour : celle ou celui qui travaille sans binôme

Vous êtes le développeur du projet. Le client veut une liste d'emprunts dans son catalogue. Le travail est découpé en sept tickets : vous les faites dans l'ordre, un ticket après l'autre, chacun sur sa branche.

Cette fiche dit **quoi faire**. Le **guide complet** explique **pourquoi** : ouvrez-le quand un morceau de code ou une commande vous échappe.

Vos tickets

Ticket	Ce que vous faites
BIB-0	Le README du projet : votre premier aller-retour avec Git
BIB-1	Le contexte de connexion
BIB-2	Les composants lisent la connexion
BIB-3	Le reducer et le contexte de la liste
BIB-4	Le bouton, le compteur, la page « Ma liste »
BIB-5	Le bouton sur la fiche d'un livre
BIB-6	La liste survit au rechargement
BIB-7	Le compte affiche la liste

Pour commencer : Git

1 · Votre poste

Dans le terminal de VS Code :

```
git --version
git config --global user.name "Prénom Nom"
git config --global user.email "vous@exemple.fr"
```

Pas de numéro de version ? Installez Git : **git-scm.com** sous Windows, `xcode-select --install` sous macOS. Il vous faut aussi un compte sur **github.com**.

---

2 · Le dépôt

Ouvrez le zip, et le dossier `catalogue-react/` dans VS Code.

```
git init
git add .
git commit -m "Code de départ"
git branch -M main
```

Sur GitHub : +, *New repository*, nom `catalogue-react`, *Public*, rien d'autre de coché.

```
git remote add origin https://github.com/VOTRE-IDENTIFIANT/catalogue-react.git
git push -u origin main
```

Vous devez voir : dans le terminal, une ligne qui finit par `main -> main`. Sur GitHub, vos fichiers apparaissent.

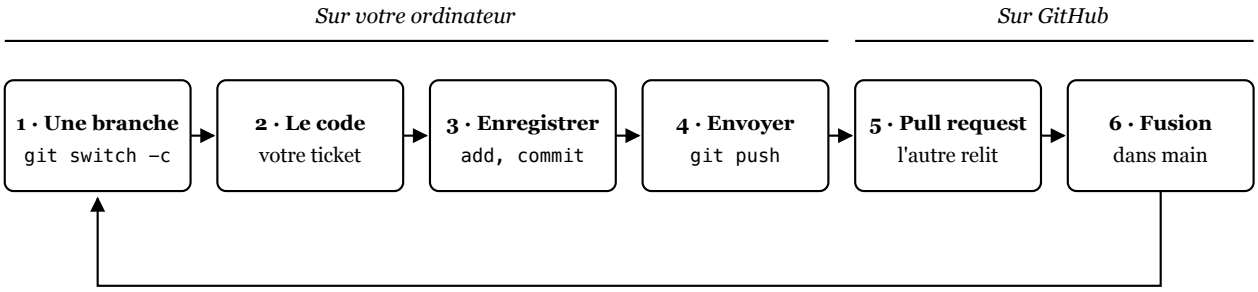
Puis lancez le projet :

```
npm install
npm run dev
```

Vous devez voir : le catalogue, à l'adresse http://localhost:5173.

3 · La boucle d'un ticket

La même pour tous vos tickets. Gardez cette page sous les yeux.



La boucle d'un ticket : trois gestes sur votre ordinateur, trois sur GitHub

Quand	Quoi
Avant le ticket	`git switch main`, `git pull`, `git switch -c LA-BRANCHE`
Le ticket est fini	`git add .`, `git commit -m "LE MESSAGE"`, `git push -u origin LA-BRANCHE`
Sur GitHub	*Compare & pull request*, puis *Create pull request*
Vous vous relisez	onglet *Files changed* : est-ce bien ce que vous vouliez envoyer, et rien d'autre ?
Vous fusionnez	*Merge pull request*, puis *Delete branch*
Retour chez vous	`git switch main`, `git pull`

Dans une équipe, c'est un collègue qui relit votre pull request. Aujourd'hui, c'est vous : prenez le temps de la lire avant de fusionner.

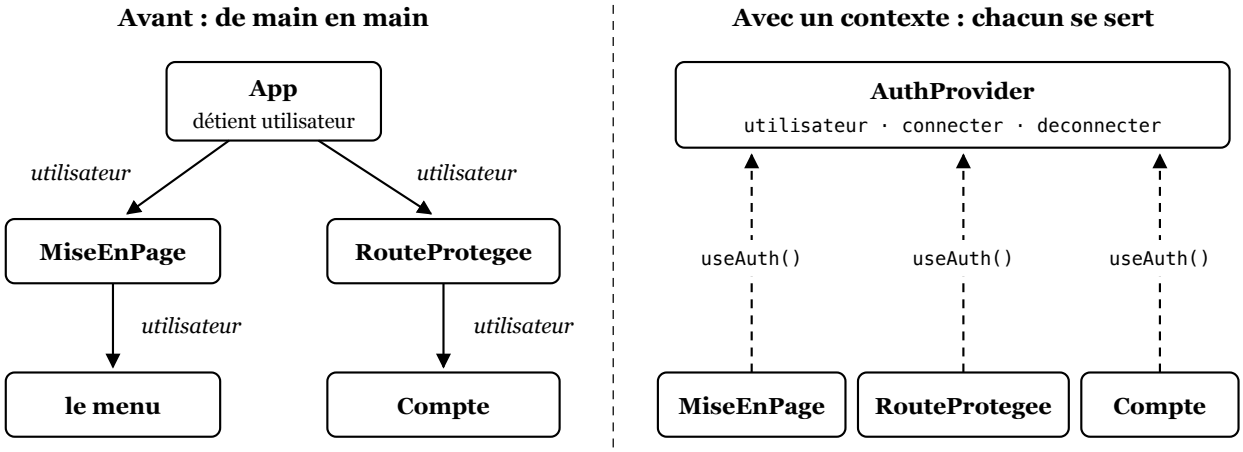
BIB-0 · Le premier aller-retour

Branche : `bib-0-readme`. Message : `BIB-0 : l'équipe et le dépôt`.

Dans `README.md`, complétez votre nom et l'adresse du dépôt. Puis faites toute la boucle.

Vous devez voir : sur GitHub, une pull request fusionnée, et le README à jour sur la branche `main`.

La connexion



Ce que changent ces deux tickets : l'utilisateur ne descend plus de main en main

---

BIB-1 · Le contexte de connexion

Branche : `bib-1-contexte-connexion`. Message : `BIB-1 : le contexte de connexion`.

Le fichier `src/contexts/AuthContext.jsx` contient un bouchon, un faux fichier provisoire : vous le remplacez.

Remplacez tout le contenu de `src/contexts/AuthContext.jsx` par :

```
import { createContext, useContext, useState } from 'react';

// Qui est connecté : une valeur que toute l'application peut lire, sans la passer
de main en main.
const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [utilisateur, setUtilisateur] = useState(null);

  const valeur = {
    utilisateur,
    connecter: setUtilisateur,
    deconnecter: () => setUtilisateur(null),
  };

  return <AuthContext value={valeur}>{children}</AuthContext>;
}

export function useAuth() {
  const contexte = useContext(AuthContext);
  if (!contexte) {
    throw new Error('useAuth doit être utilisé dans un <AuthProvider>.');
  }
  return contexte;
}
```

Dans `src/main.jsx`, juste après la ligne `import { BrowserRouter } from 'react-router';` :

```
import { AuthProvider } from './contexts/AuthContext.jsx';
```

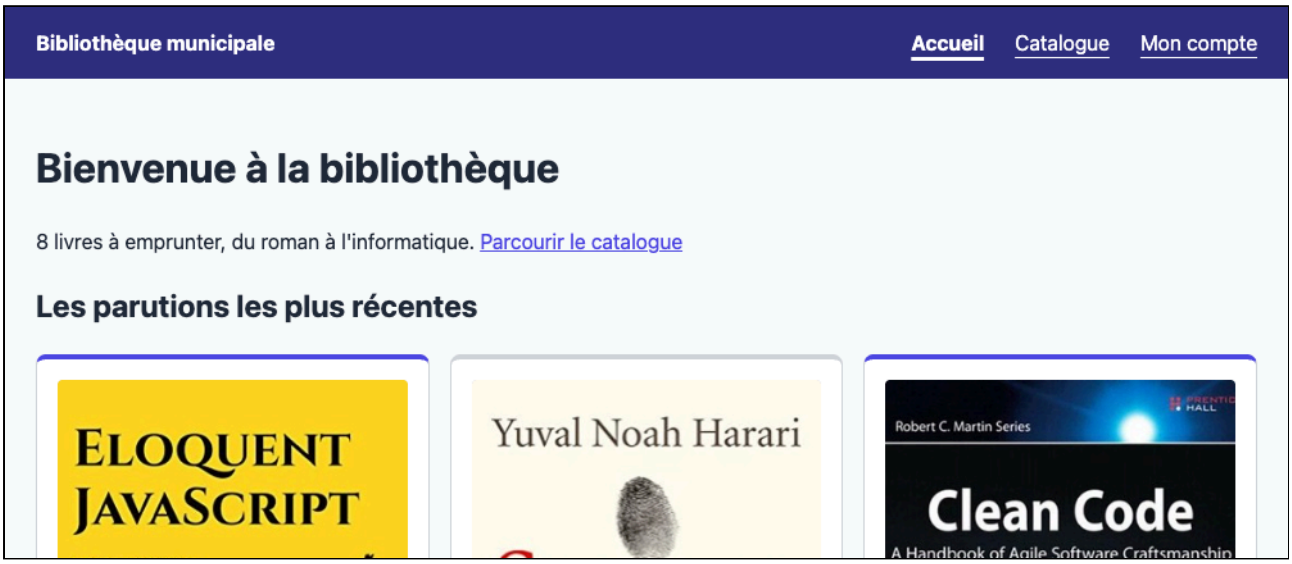
Dans `src/main.jsx`, remplacez :

```
      <App />
```

par :

```
      <AuthProvider>
        <App />
      </AuthProvider>
```

Vous devez voir : un `AuthProvider` au-dessus d'`App`, dans l'onglet *Components*. À l'écran, rien ne change, et c'est normal.



BIB-1 : l'application s'affiche comme avant

Livrez le ticket : toute la boucle, jusqu'au `git pull`.

BIB-2 · Les composants lisent la connexion

Branche : `bib-2-lire-la-connexion`. Message : `BIB-2 : les composants lisent la connexion`.

Remplacez tout le contenu de `src/App.jsx` par :

```
import { Routes, Route } from 'react-router';
import MiseEnPage from './components/MiseEnPage.jsx';
import RouteProtegee from './components/RouteProtegee.jsx';
import Accueil from './pages/Accueil.jsx';
import Catalogue from './pages/Catalogue.jsx';
import FicheLivre from './pages/FicheLivre.jsx';
import Connexion from './pages/Connexion.jsx';
import Compte from './pages/Compte.jsx';
import PageIntrouvable from './pages/PageIntrouvable.jsx';

// Les adresses de l'application : une route = une adresse = une page.
export default function App() {
  return (
    <Routes>
      <Route element={<MiseEnPage />}>
        <Route index element={<Accueil />} />
        <Route path="catalogue" element={<Catalogue />} />
        <Route path="livres/:id" element={<FicheLivre />} />
        <Route path="connexion" element={<Connexion />} />
        <Route
          path="compte"
          element={
            <RouteProtegee>
              <Compte />
            </RouteProtegee>
          }
        />
        <Route path="*" element={<PageIntrouvable />} />
      </Route>
    </Routes>
  );
}
```

La mise en page.

Dans `src/components/MiseEnPage.jsx`, juste après la ligne `import { Link, NavLink, Outlet } from 'react-router';` :

```
import { useAuth } from '../contexts/AuthContext.jsx';
```

Dans `src/components/MiseEnPage.jsx`, remplacez :

```
export default function MiseEnPage({ utilisateur, onDeconnexion }) {
```

par :

```
export default function MiseEnPage() {
  const { utilisateur, deconnecter } = useAuth();
```

Dans `src/components/MiseEnPage.jsx`, remplacez :

```
                <button type="button" onClick={onDeconnexion}>Se
déconnecter</button>
```

par :

```
                <button type="button" onClick={deconnecter}>Se déconnecter</button>
```

La garde.

Dans `src/components/RouteProtegee.jsx`, juste après la ligne `import { Navigate, useLocation } from 'react-router';` :

```
import { useAuth } from '../contexts/AuthContext.jsx';
```

Dans `src/components/RouteProtegee.jsx`, remplacez :

```
export default function RouteProtegee({ utilisateur, children }) {
```

par :

```
export default function RouteProtegee({ children }) {
  const { utilisateur } = useAuth();
```

La connexion.

Dans `src/pages/Connexion.jsx`, juste après la ligne `import { adherents } from '../data/adherents.js';` :

```
import { useAuth } from '../contexts/AuthContext.jsx';
```

Dans `src/pages/Connexion.jsx`, remplacez :

```
export default function Connexion({ onConnexion }) {
```

par :

```
export default function Connexion() {
  const { connecter } = useAuth();
```

Dans `src/pages/Connexion.jsx`, remplacez :

```
    onConnexion({ id: adherent.id, nom: adherent.nom, email: adherent.email });
```

par :

```
    connecter({ id: adherent.id, nom: adherent.nom, email: adherent.email });
```

Le compte.

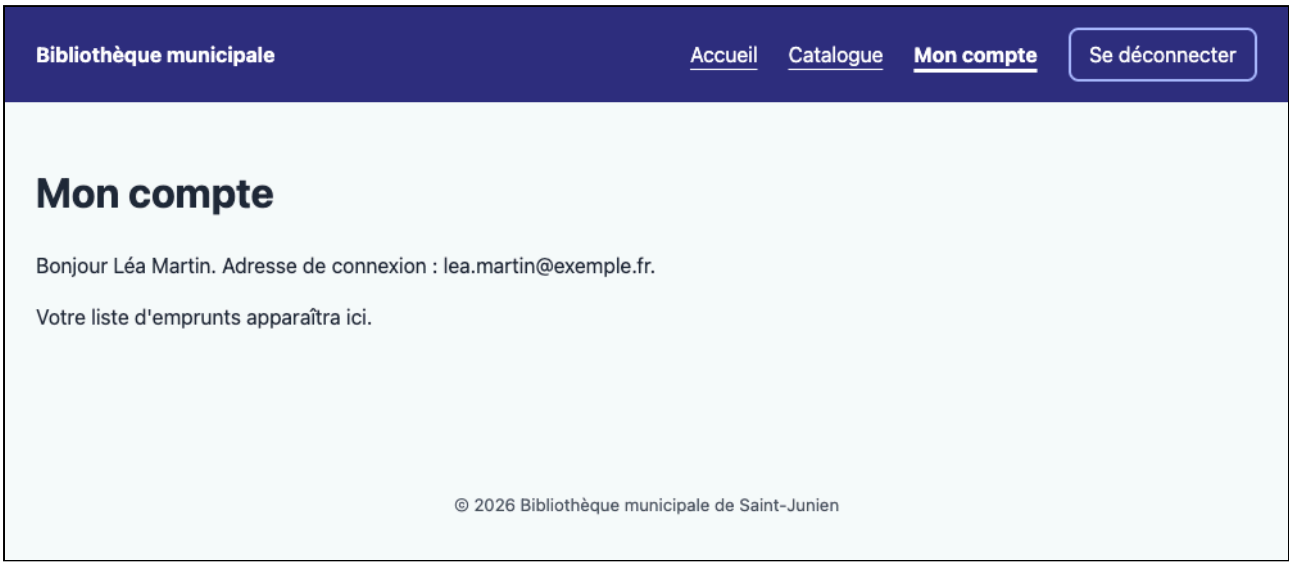
Remplacez tout le contenu de `src/pages/Compte.jsx` par :

```
import { useAuth } from '../contexts/AuthContext.jsx';

export default function Compte() {
  const { utilisateur } = useAuth();

  return (
    <>
      <title>Mon compte — Bibliothèque municipale</title>
      <h1>Mon compte</h1>
      <p>Bonjour {utilisateur.nom}. Adresse de connexion : {utilisateur.email}.</p>
      <p>Votre liste d'emprunts apparaîtra ici.</p>
    </>
  );
}
```

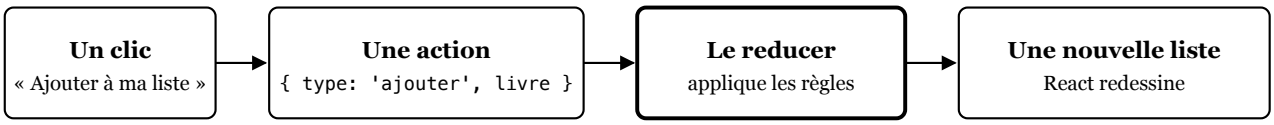
Vous devez voir : connectez-vous avec `lea.martin@exemple.fr`, mot de passe `lecture`. La connexion marche, et « Mon compte » s'ouvre.



BIB-2 : Léa est connectée, le menu propose « Se déconnecter »

Livrez le ticket.

La liste



Ce que fait le reducer, à chaque clic

BIB-3 · Le reducer et le contexte de la liste

Branche : `bib-3-reducer-liste`. Message : `BIB-3 : le reducer et le contexte de la liste`.

Les deux fichiers sont des bouchons : vous les remplacez.

Remplacez tout le contenu de `src/contexts/listeReducer.js` par :

```
// La liste d'emprunts : au plus 3 livres, chacun une seule fois.
export const MAXIMUM = 3;

// Un reducer : (état actuel, action) → nouvel état. Il ne modifie rien, il ne fait
rien d'autre.
export function listeReducer(liste, action) {
  switch (action.type) {
    case 'ajouter': {
      const dejaLa = liste.some((l) => l.id === action.livre.id);
      if (dejaLa || liste.length >= MAXIMUM) {
        return liste;
      }
      const { id, titre, auteur } = action.livre;
      return [...liste, { id, titre, auteur }];
    }
    case 'retirer':
      return liste.filter((l) => l.id !== action.id);
    case 'vider':
      return [];
    default:
      throw new Error(`Action inconnue : ${action.type}`);
  }
}
```

Remplacez tout le contenu de `src/contexts/ListeContext.jsx` par :

```
import { createContext, useContext, useReducer } from 'react';
import { listeReducer, MAXIMUM } from './listeReducer.js';

const ListeContext = createContext(null);

export function ListeProvider({ children }) {
  const [liste, dispatch] = useReducer(listeReducer, []);

  const valeur = {
    liste,
    pleine: liste.length >= MAXIMUM,
    contient: (id) => liste.some((l) => l.id === id),
    ajouter: (livre) => dispatch({ type: 'ajouter', livre }),
    retirer: (id) => dispatch({ type: 'retirer', id }),
    vider: () => dispatch({ type: 'vider' }),
  };

  return <ListeContext value={valeur}>{children}</ListeContext>;
}

export function useListe() {
  const contexte = useContext(ListeContext);
  if (!contexte) {
    throw new Error('useListe doit être utilisé dans un <ListeProvider>.');
  }
  return contexte;
}
```

Dans `src/main.jsx`, juste après la ligne `import { AuthProvider } from './contexts/AuthContext.jsx';` :

```
import { ListeProvider } from './contexts/ListeContext.jsx';
```

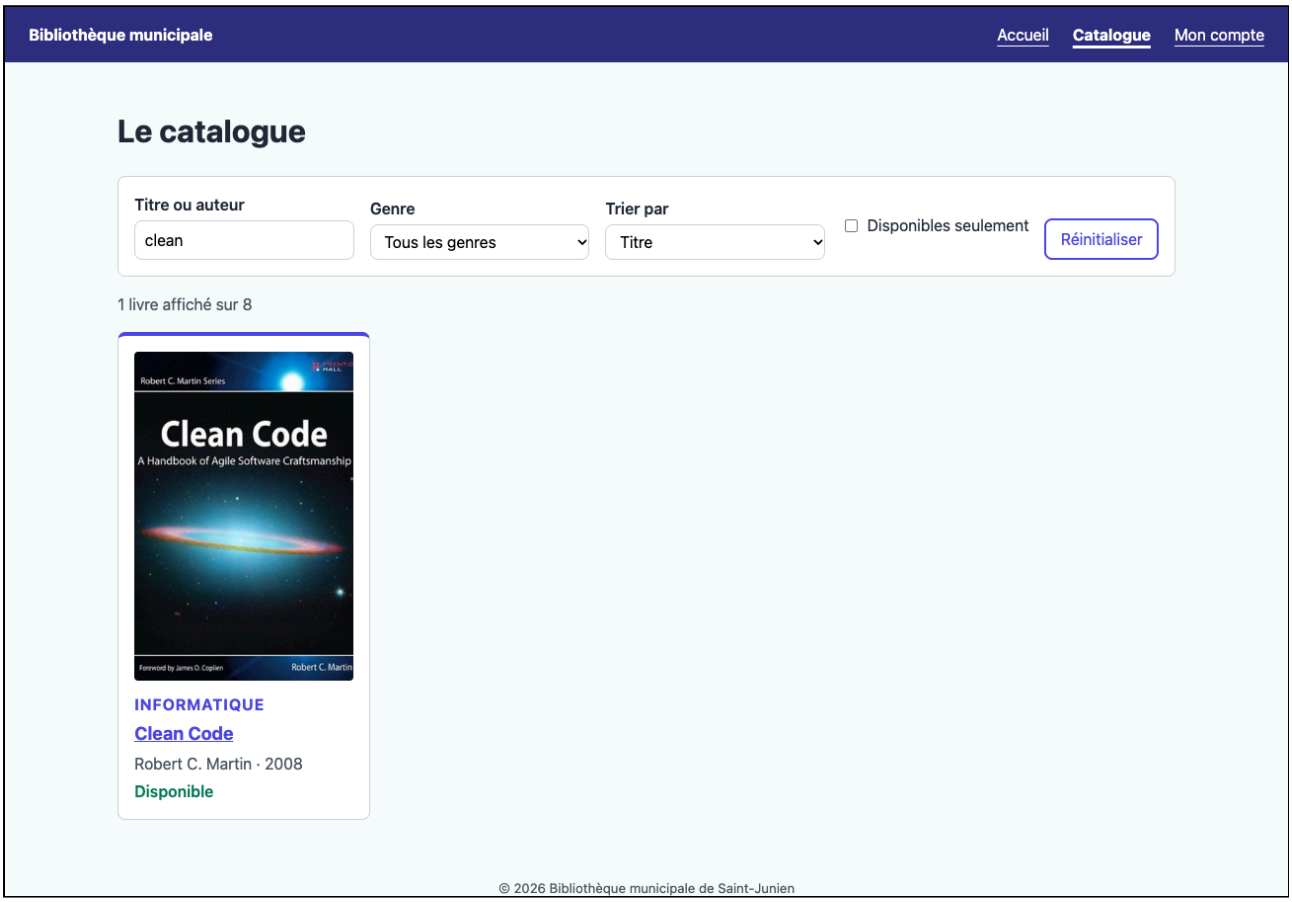
Dans `src/main.jsx`, remplacez :

```
        <App />
```

par :

```
        <ListeProvider>
          <App />
        </ListeProvider>
```

Vous devez voir : un `ListeProvider` sous l'`AuthProvider`, dans l'onglet *Components*. À l'écran, pas encore de bouton : c'est le ticket suivant.



BIB-3 : le catalogue n'a pas encore de bouton

Livrez le ticket.

BIB-4 · Le bouton, le compteur, la page

Branche : `bib-4-bouton-compteur-page`. Message : `BIB-4 : le bouton, le compteur et la page Ma liste`.

Créez `src/components/BoutonListe.jsx` :

```
import { useListe } from '../contexts/ListeContext.jsx';

// Le même bouton sur les cartes et sur la fiche : il sait tout seul dans quel état
il est.
export default function BoutonListe({ livre }) {
  const { contient, pleine, ajouter } = useListe();

  if (!livre.disponible) {
    return <button type="button" disabled>Indisponible</button>;
  }
  if (contient(livre.id)) {
    return <button type="button" disabled>Dans ma liste</button>;
  }
  if (pleine) {
    return <button type="button" disabled>Liste pleine</button>;
  }
  return (
    <button type="button" onClick={() => ajouter(livre)}>
      Ajouter à ma liste
    </button>
  );
}
```

Dans `src/components/CarteLivre.jsx`, juste après la ligne `import { Link } from 'react-router';` :

```
import BoutonListe from './BoutonListe.jsx';
```

Dans `src/components/CarteLivre.jsx`, juste après la ligne `<p className="dispo">{livre.disponible ? 'Disponible' : 'Emprunté'}</p>` :

```
      <div className="actions">
        <BoutonListe livre={livre} />
      </div>
```

Dans `src/components/MiseEnPage.jsx`, juste après la ligne `import { useAuth } from '../contexts/AuthContext.jsx';` :

```
import { useListe } from '../contexts/ListeContext.jsx';
```

Dans `src/components/MiseEnPage.jsx`, juste après la ligne `const { utilisateur, deconnecter } = useAuth();` :

```
  const { liste } = useListe();
```

Dans `src/components/MiseEnPage.jsx`, juste après la ligne `<li><NavLink to="/catalogue">Catalogue</NavLink></li>` :

```
            <li>
              <NavLink to="/ma-liste">
                Ma liste <span className="pastille">{liste.length}</span>
              </NavLink>
            </li>
```

Créez `src/pages/MaListe.jsx` :

```
import { Link } from 'react-router';
import { useListe } from '../contexts/ListeContext.jsx';
import { MAXIMUM } from '../contexts/listeReducer.js';

export default function MaListe() {
  const { liste, retirer, vider } = useListe();
  const s = liste.length > 1 ? 's' : '';

  function confirmerVider() {
    if (window.confirm('Retirer tous les livres de votre liste ?')) {
      vider();
    }
  }

  return (
    <>
      <title>Ma liste — Bibliothèque municipale</title>
      <h1>Ma liste d'emprunts</h1>

      {liste.length === 0 ? (
        <p className="aucun">
          Votre liste est vide. <Link to="/catalogue">Choisir des livres dans le
catalogue</Link>
        </p>
      ) : (
        <>
          <p className="compteur">{liste.length} livre{s} sur {MAXIMUM} possibles.
</p>
          <ul className="selection">
            {liste.map((livre) => (
              <li key={livre.id}>
                <p><Link to={`/livres/${livre.id}`}>{livre.titre}</Link>,
{livre.auteur}</p>
                <button
                  type="button"
                  className="secondaire"
                  aria-label={`Retirer ${livre.titre}`}
                  onClick={() => retirer(livre.id)}
                >
```

```
                Retirer
              </button>
            </li>
          ))}
        </ul>
        <p className="barre">
          <button type="button" className="secondaire" onClick={confirmerVider}>
            Vider la liste
          </button>
        </p>
      </>
    )}
  </>
);
}
```

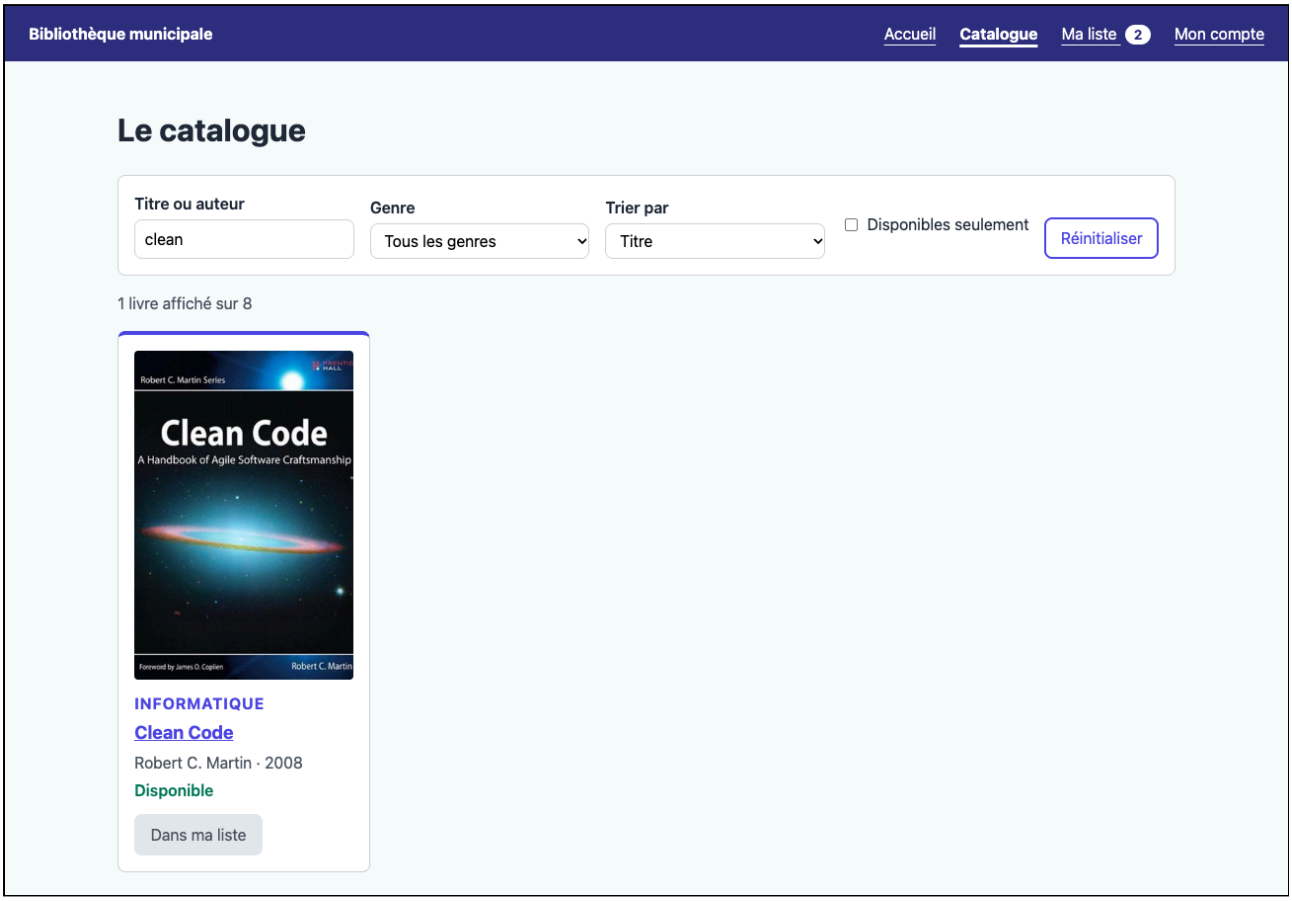
Dans `src/App.jsx`, juste après la ligne `import FicheLivre from './pages/FicheLivre.jsx';` :

```
import MaListe from './pages/MaListe.jsx';
```

Dans `src/App.jsx`, juste après la ligne `<Route path="livres/:id" element={<FicheLivre />} />` :

```
      <Route path="ma-liste" element={<MaListe />} />
```

Vous devez voir : ajoutez deux livres, le compteur du menu passe à 2. Dans « Ma liste », retirez-en un, puis videz la liste.



BIB-4 : deux livres ajoutés, le compteur dit 2

Livrez le ticket.

BIB-5 · Le bouton sur la fiche

Branche : `bib-5-bouton-fiche`. Message : `BIB-5 : le bouton sur la fiche`.

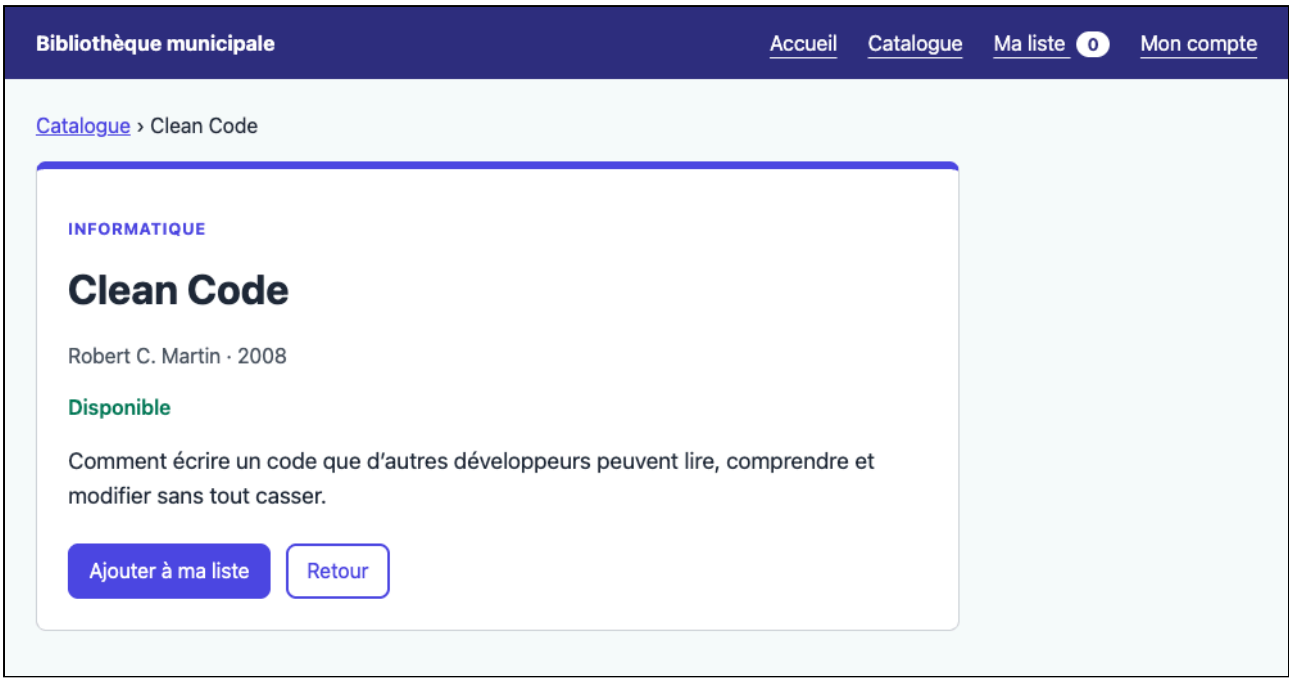
Dans `src/pages/FicheLivre.jsx`, juste après la ligne `import { livres } from '../data/livres.js';` :

```
import BoutonListe from '../components/BoutonListe.jsx';
```

Dans `src/pages/FicheLivre.jsx`, juste après la ligne `<div className="actions">` :

```
          <BoutonListe livre={livre} />
```

Vous devez voir : sur la fiche de *Clean Code*, le même bouton que sur sa carte.



BIB-5 : le bouton sur la fiche d'un livre

Livrez le ticket.

La mémoire

BIB-6 · La liste survit au rechargement

Branche : `bib-6-memoire`. Message : `BIB-6 : la liste survit au rechargement`.

Dans `src/contexts/ListeContext.jsx`, remplacez :

```
import { createContext, useContext, useReducer } from 'react';
```

par :

```
import { createContext, useContext, useEffect, useReducer } from 'react';
```

Dans `src/contexts/ListeContext.jsx`, juste après la ligne `const ListeContext = createContext(null);` :

```
const CLE = 'liste-emprunts';

// Au premier affichage, on relit la liste enregistrée ; si elle est absente ou abîmée, liste vide.
function charger() {
  try {
    return JSON.parse(localStorage.getItem(CLE)) ?? [];
  } catch {
    return [];
  }
}
```

Dans `src/contexts/ListeContext.jsx`, remplacez :

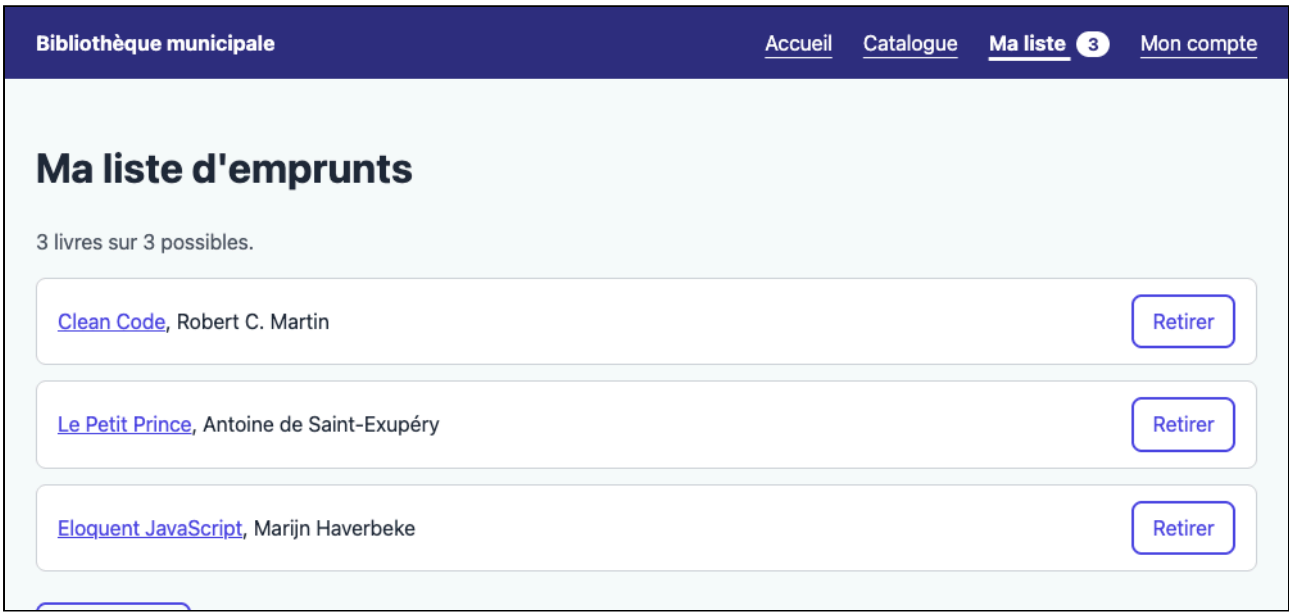
```
  const [liste, dispatch] = useReducer(listeReducer, []);
```

par :

```
const [liste, dispatch] = useReducer(listeReducer, [], charger);

// Après chaque changement de la liste, on l'enregistre.
useEffect(() => {
  localStorage.setItem(CLE, JSON.stringify(liste));
}, [liste]);
```

Vous devez voir : trois livres dans la liste, `F5`, ils sont toujours là.



BIB-6 : après un rechargement, la liste est intacte

Livrez le ticket.

BIB-7 · Le compte affiche la liste

Branche : `bib-7-compte`. Message : `BIB-7 : le compte affiche la liste`.

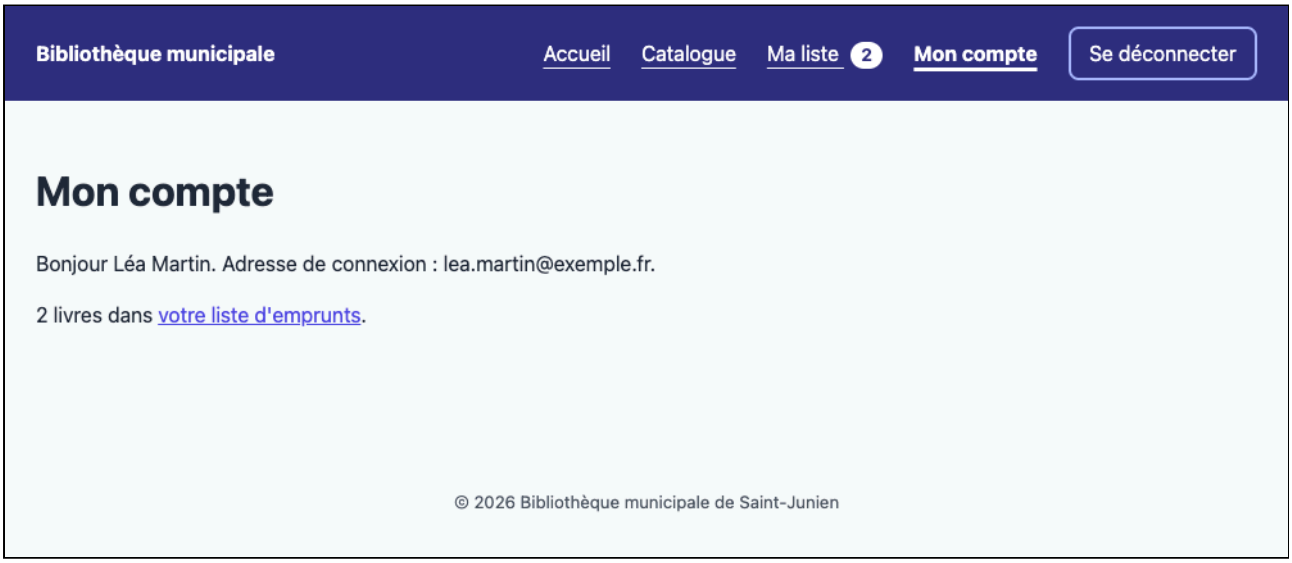
Remplacez tout le contenu de `src/pages/Compte.jsx` par :

```
import { Link } from 'react-router';
import { useAuth } from '../contexts/AuthContext.jsx';
import { useListe } from '../contexts/ListeContext.jsx';

export default function Compte() {
  const { utilisateur } = useAuth();
  const { liste } = useListe();
  const s = liste.length > 1 ? 's' : '';

  return (
    <>
      <title>Mon compte — Bibliothèque municipale</title>
      <h1>Mon compte</h1>
      <p>Bonjour {utilisateur.nom}. Adresse de connexion : {utilisateur.email}.</p>
      <p>
        {liste.length} livre{s} dans <Link to="/ma-liste">votre liste
d'emprunts</Link>.
      </p>
    </>
  );
}
```

Vous devez voir : connecté, avec deux livres, « 2 livres dans votre liste d'emprunts ».



BIB-7 : le compte rappelle le nombre de livres

Livrez le ticket.

Rendre

Sur la page du dépôt GitHub : *Code*, puis *Download ZIP*. Renommez le fichier en `NOM-Prenom-React-S3.zip`, et déposez-le dans **Rendus — à déposer ici → Frameworks React → FM04 · Séance 3**.

Vous avez fini ? Le TP bonus, « La ludothèque garde vos favoris », reprend les mêmes notions sur un autre projet.

BC01-FM04 · séance 3 · mardi 10 novembre 2026, matin · J. Macadre